A problematic timing

consider the following sequence of lock()

```
queue_add(m->q, gettid()); // thread joins queue
m->guard = 0;              // releases guard
park();                    // thread attempts to sleep
```

Lets call this thread T1. park() is intended to block T1 until another thread calls unpark(T1).

But what if scheduler preempts T1 right before T1 calls park().

Another thread, T2, holding the lock, executes unlock and unpark(T_1).

→ T_1 is removed from the queue and unparked, but is still preempted and hasn't executed park() yet.

Again, when T1 resumes and calls park()

→ it sleeps forever! → wake up/waiting race.

Different systems fixes the above problem in different ways.

eg. - Solaris solves it by adding a new system call setpark() - indicating, it is about to park.

so that if thread gets preempted as and another thread calls unpark before park, its subsequent park returns immediately instead of sleeping.

```
1        queue_add(m->q, gettid());
2        setpark(); // new code
3        m->guard = 0;
```

Two-phase locks: a hybrid approach

As its name indicates, such a lock has two phases.

Phase-1 → lock spins for a while, hoping that it can acquire the lock.

Phase-2 → If the lock is not acquired during Phase-1, here the caller is put to sleep, and only woken up when the lock becomes free later.

eg:- Linux Futex lock

Lock-based concurrent data structures

Question: How to add locks to data structures such that it works concurrently maintaining high performance?

Concurrent counters

```
1  typedef struct __counter_t {
2      int value;
3  } counter_t;
4
5  void init(counter_t *c) {
6      c->value = 0;
7  }
8
9  void increment(counter_t *c) {
10     c->value++;
11 }
12
13 void decrement(counter_t *c) {
14     c->value--;
15 }
16
17 int get(counter_t *c) {
18     return c->value;
19 }
```

→ a counter without locks.
(non-synchronized)

Question: How can we make this code thread safe?

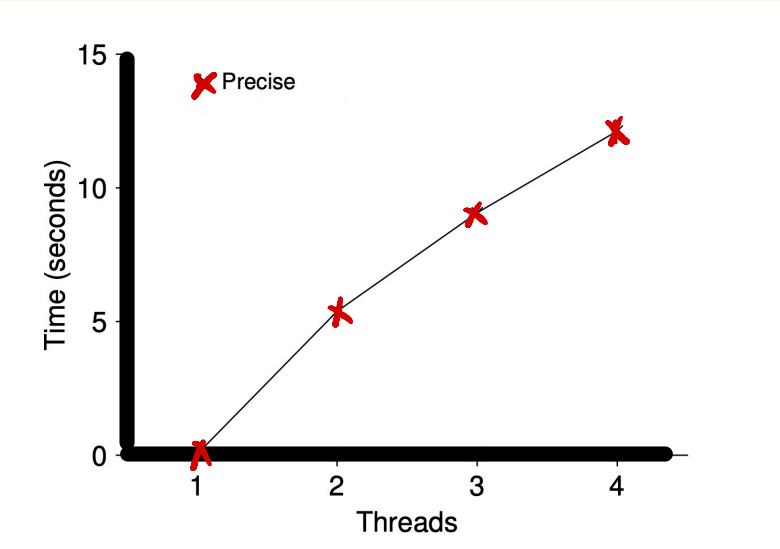
→ add a single lock, which needs to be acquired whenever a calling routine tries to access the data structure.

```c
typedef struct __counter_t {
    int             value;
    pthread_mutex_t lock;
} counter_t;

void init(counter_t *c) {
    c->value = 0;
    Pthread_mutex_init(&c->lock, NULL);
}

void increment(counter_t *c) {
    Pthread_mutex_lock(&c->lock);
    c->value++;
    Pthread_mutex_unlock(&c->lock);
}

void decrement(counter_t *c) {
    Pthread_mutex_lock(&c->lock);
    c->value--;
    Pthread_mutex_unlock(&c->lock);
}

int get(counter_t *c) {
    Pthread_mutex_lock(&c->lock);
    int rc = c->value;
    Pthread_mutex_unlock(&c->lock);
    return rc;
}
```

a counter with locks

What about the performance cost?

↳ scales poorly.



* each thread updates the counter 1 million times

(run upon an iMac with four Intel 2.7 GHz i5 CPUs)

- when a single thread can complete the million counter updates in roughly 0.03 seconds, even with just 2 threads running concurrently, there is a massive slow down. gets worse with more threads!

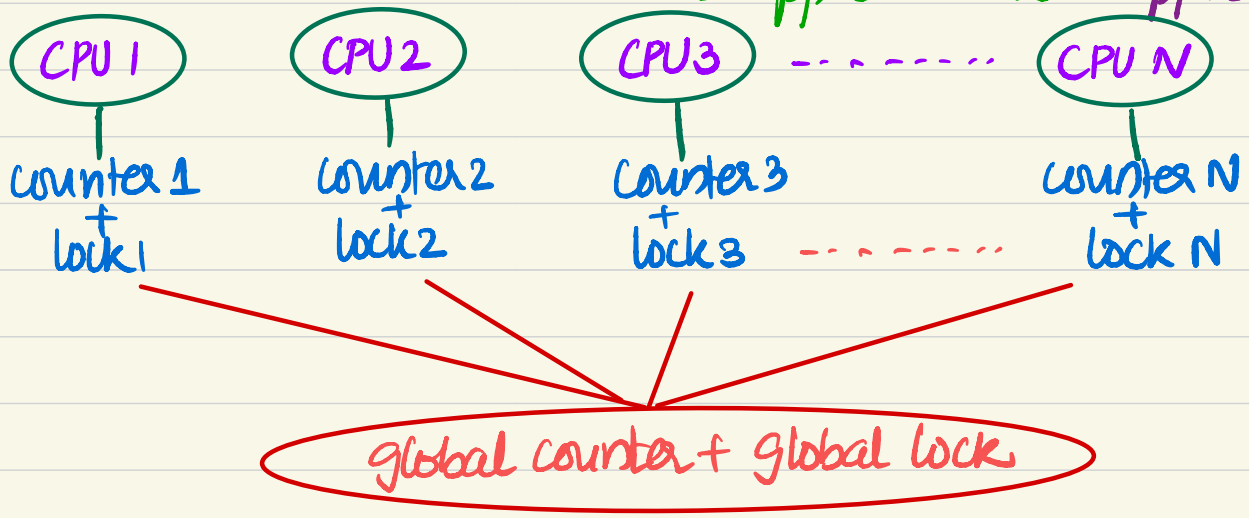
Goal: To complete threads just as quickly on multiprocessors as the single thread does on one.

→ perfect scaling.

Question: How to achieve a scalable counting?

→ one approach is approximate counter.

Idea:



- when a thread running on a given core wishes to increment the counter, it increments its local counter

Whose access is synchronized via corresponding local clock.

What about the updation of the global counter and how?

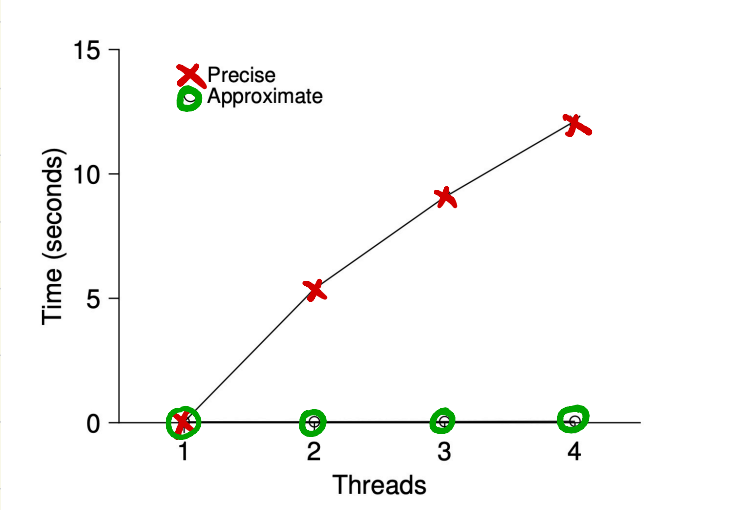
→ periodically transfer the local values to the global counter, by acquiring the global lock → increment it by local counter's value

How often to do this transfer? → determined by a threshold S
small S ⇒ less scalable but global value closer to actual count
large S ⇒ more " " " further off from "

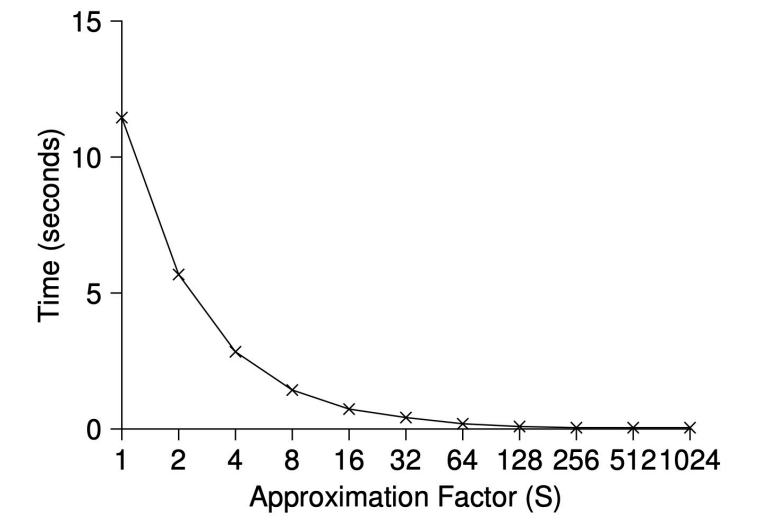
Time	L_1	L_2	L_3	L_4	G
0	0	0	0	0	0
1	0	0	1	1	0
2	1	0	2	1	0
3	2	0	3	1	0
4	3	0	3	2	0
5	4	1	3	3	0
6	$5 \rightarrow 0$	1	3	4	5 (from L_1)
7	0	2	4	$5 \rightarrow 0$	10 (from L_4)

the transfer is done whenever a local counter reaches the threshold.

Tracing the approximate counters threshold, S = 5



Performance: traditional Vs Approximate counters (S = 1024)



scaling approximate counters (the choice of S is important for the scaling).

```c
typedef struct __counter_t {
    int             global;          // global count
    pthread_mutex_t glock;           // global lock
    int             local[NUMCPUS]; // per-CPU count
    pthread_mutex_t llock[NUMCPUS]; // ... and locks
    int             threshold;       // update freq
} counter_t;

// init: record threshold, init locks, init values
//       of all local counts and global count
void init(counter_t *c, int threshold) {
    c->threshold = threshold;
    c->global = 0;
    pthread_mutex_init(&c->glock, NULL);
    int i;
    for (i = 0; i < NUMCPUS; i++) {
        c->local[i] = 0;
        pthread_mutex_init(&c->llock[i], NULL);
    }
}

// update: usually, just grab local lock and update
// local amount; once it has risen 'threshold',
// grab global lock and transfer local values to it
void update(counter_t *c, int threadID, int amt) {
    int cpu = threadID % NUMCPUS;
    pthread_mutex_lock(&c->llock[cpu]);
    c->local[cpu] += amt;
    if (c->local[cpu] >= c->threshold) {
        // transfer to global (assumes amt>0)
        pthread_mutex_lock(&c->glock);
        c->global += c->local[cpu];
        pthread_mutex_unlock(&c->glock);
        c->local[cpu] = 0;
    }
    pthread_mutex_unlock(&c->llock[cpu]);
}

// get: just return global amount (approximate)
int get(counter_t *c) {
    pthread_mutex_lock(&c->glock);
    int val = c->global;
    pthread_mutex_unlock(&c->glock);
    return val; // only approximate!
}
```

set threshold

other initializations

pick a local slot to use

lock local slot and update local counter

→ reached threshold → transfer!

lock global counter and add local to global

reset local counter

returns the global count only, ignoring local values that haven't yet been flushed

→ Hence, it's approximate.

Concurrent Linked Lists

→ a thread-safe linked list, implemented using a single global lock per list.

```c
// basic node structure
typedef struct __node_t {
    int                 key;
    struct __node_t     *next;
} node_t;

// basic list structure (one used per list)
typedef struct __list_t {
    node_t              *head;
    pthread_mutex_t     lock;
} list_t;

void List_Init(list_t *L) {
    L->head = NULL;
    pthread_mutex_init(&L->lock, NULL);
}

int List_Insert(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *new = malloc(sizeof(node_t));
    if (new == NULL) {
        perror("malloc");
        pthread_mutex_unlock(&L->lock);
        return -1; // fail
    }
    new->key  = key;
    new->next = L->head;
    L->head   = new;
    pthread_mutex_unlock(&L->lock);
    return 0; // success
}

int List_Lookup(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *curr = L->head;
    while (curr) {
        if (curr->key == key) {
            pthread_mutex_unlock(&L->lock);
            return 0; // success
        }
        curr = curr->next;
    }
    pthread_mutex_unlock(&L->lock);
    return -1; // failure
}
```

Goal: Allow multiple threads to insert/look up elements safely (i.e., no race conditions)

→ - lock the list → only one thread can insert at a time.
- allocate a new node
- insert at the head
- unlock the list

Thus correctness is ensured in insertion, (as no two threads can modify head concurrently).

→ lock the list for the entire search
- traverse the linked list node by node

if found → unlock and return success

If not found → unlock and return failure

- Note that no other thread can modify the list while you're traversing.
- avoids reading a node that might be freed or inserted concurrently -

Some tricky scenarios: Recall the following part of the code.

```
int List_Insert(list_t *L, int key) {
    pthread_mutex_lock(&L->lock);
    node_t *new = malloc(sizeof(node_t));
    if (new == NULL) {
        perror("malloc");
        pthread_mutex_unlock(&L->lock);
        return -1;
    }
    ...
    pthread_mutex_unlock(&L->lock);
    return 0;
}
```

There are two exit paths here:
- one for failure (malloc fails)
- one for success (normal insert)

and each path has to unlock before returning.

- If you forget to unlock one of those paths, you cause a deadlock!
(think, why?)

In general, when you have multiple return points, it's easy to make a mistake.

Question: can we rewrite insert and lookup to remain correct under concurrent insert but avoid the failure path also requires to call unlock? YES,

Infact, malloc() and key initialization can be done outside the critical section.

(Why?, because memory allocation and local variable set up don't touch shared data so, we don't need to hold the lock yet – other threads can safely run while this is happening).

- only the actual shared modification (L->head) is done under lock. This narrows down the critical section
  → better performance

- no chance of forgetting to unlock (as there is no failure path inside the locked region).

EXERCISE
Read concurrent queues and concurrent hashtables.

```
1  void List_Init(list_t *L) {
2      L->head = NULL;
3      pthread_mutex_init(&L->lock, NULL);
4  }
5
6  int List_Insert(list_t *L, int key) {
7      // synchronization not needed
8      node_t *new = malloc(sizeof(node_t));
9      if (new == NULL) {
10         perror("malloc");
11         return -1;
12     }
13     new->key = key;
14     // just lock critical section
15     pthread_mutex_lock(&L->lock);
16     new->next = L->head;
17     L->head   = new;
18     pthread_mutex_unlock(&L->lock);
19     return 0; // success
20 }
21
22 int List_Lookup(list_t *L, int key) {
23     int rv = -1;
24     pthread_mutex_lock(&L->lock);
25     node_t *curr = L->head;
26     while (curr) {
27         if (curr->key == key) {
28             rv = 0;
29             break;
30         }
31         curr = curr->next;
32     }
33     pthread_mutex_unlock(&L->lock);
34     return rv; // now both success and failure
35 }
```